2_10_marzo

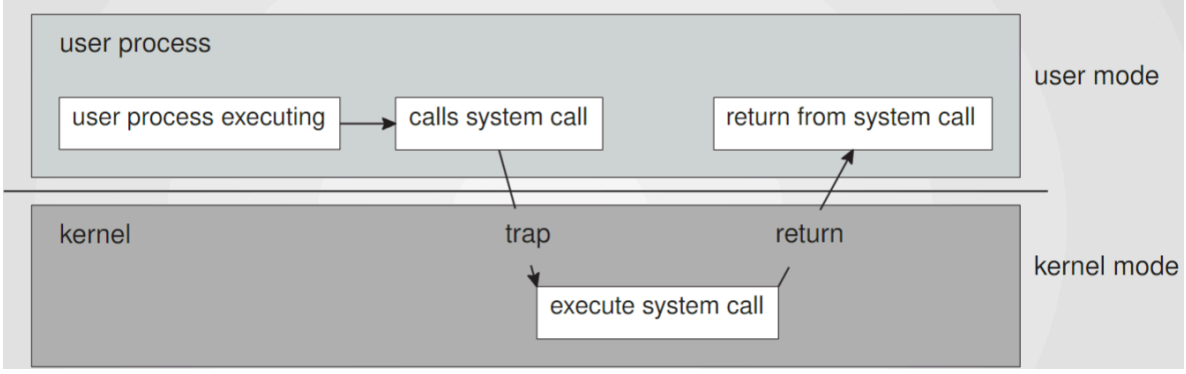
Definizioni:
Gli utenti del sistema operativo sono i processi(system call);

Le librerie sono raccolte di codice già scritto e riutilizzabile che possiamo utilizzare, ad esempio anziché andare a costruirsi una struttura dati (heap, pila, lista ecc...) possiamo usare questi sistemi già pronti che altri hanno già creato in precedenza (il vantaggio oltre al tempo risparmiato e ad avere un minor numero di errori è che le istruzioni di libreria sono ottimizzate in modo da avere una complessità il più bassa possibile)

Una **system call** è il meccanismo con cui un programma utente chiede un servizio al sistema operativo, i programmi normali (excel, word, chrome) girano in user mode, quindi non possono accedere direttamente all'hardware o a risorse sensibili, per fare queste operazioni devono chiedere al kernel tramite una system call
Esempi di system call:

- usare hardware (tastiera, lettore dischi)
- allocare memoria
- creare processi

La **TRAP** è un eccezione usata per passare dalla user mode alla kernel mode quando si deve fare una system call generata dal programma in esecuzione



Si segue questo ordine: il programma è in esecuzione in user mode (CPU in user mode), avviene una system call (inizia la TRAP), quindi la CPU passa in kernel mode si esegue questa system call e poi si ritorna alla user mode. Quando si esegue questo switch la CPU si salva il contesto precedente alla TRAP, in modo da poter ritornare a eseguire correttamente le operazioni una volta tornata in user mode.
Cosi qualunque processo può fare richiesta al sistema operativo che poi sceglierà se soddisfarla o no.

L' **interrupt hardware** è simile ad un operazione di TRAP ma differisce per il fatto che l'interrupt è mandato dall'hardware (anziché dal software con la system call), gli interrupt sono un meccanismo che permette di notificare qualcosa alla CPU, interrompe

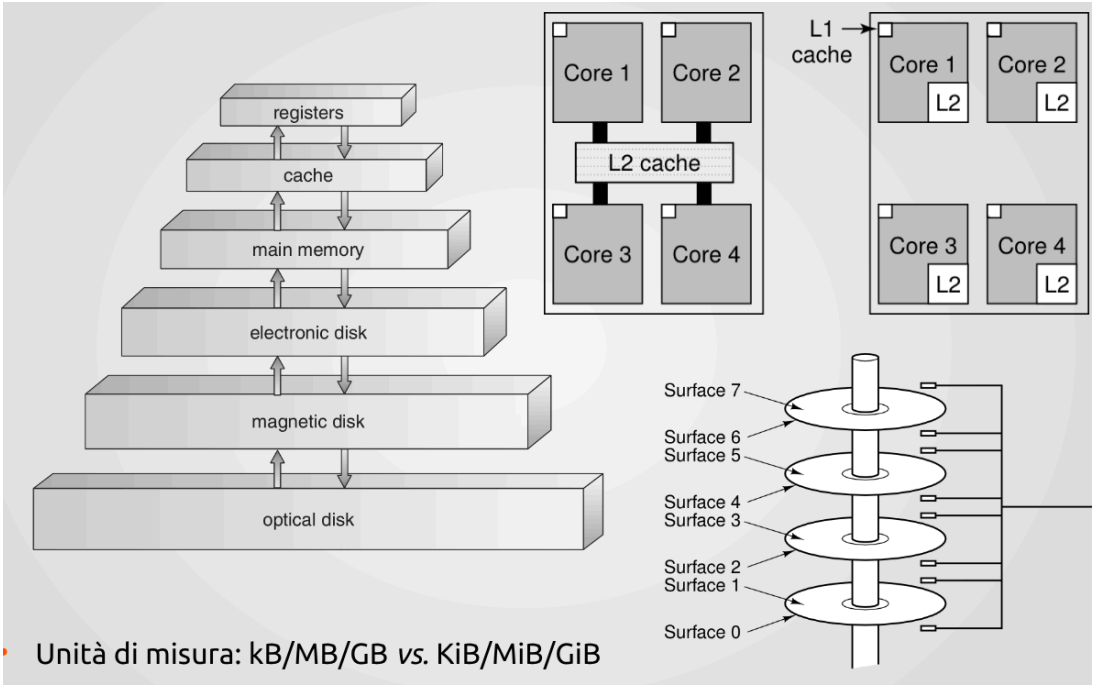
quindi quello che sta facendo in quel momento per eseguire la routine associata all'interrupt (è questo ad esempio il motivo per cui la freccia del mouse è sempre fluida senza mai bloccarsi anche quando il computer è sotto sforzo, perché il mouse lancia un interrupt e si fa riservare un piccolo spazio di esecuzione per essere eseguito senza problemi). La CPU salva il minimo indispensabile per tornare a riprendere quello che stava facendo prima (salvataggio del contesto quindi Program Counter ecc...) in modo simile alla TRAP, la CPU riprende il suo lavoro.
In generale un interrupt non deve procurare problemi, tutto poi deve continuare normalmente.

Ottimizzazione dei processi nella CPU
Tra i vari meccanismi possiamo trovare **multithreading**, si riferisce all' ottimizzazione di una CPU. Questo escamotage prevede di implementare un doppio contesto di esecuzione all'interno dello stesso core.
L'idea è: nel ciclo di decode execute ci sono tempi morti(quando la CPU deve accedere ad allocazioni vuote della RAM), nei registri vengono caricati i set di valori associati a due diversi processi: se ho il processo P1 che implica un certo numero di cicli affinché un suo fetch venga concluso, nel tempo morto verranno eseguiti degli step del P2.
Sembra una complicazione, ma permette di lavorare su processi diversi in modo più efficace, *ma questa soluzione non è parallelismo*, non ce una doppia CPU, il multithreading viene effettuato sullo stesso core. in un dato istante si attenziona o uno o l'altro processo. Il sistema operativo deve però capire che l' ALU è sempre la stessa evitando di portare a utilizzi strani della risorsa, evitando quindi di bloccare i processi.

Un altro sistema sono i **multiprocessori** (*da notare NON multicore*) in cui aggiungo più CPU in modo da poter *parallelizzare* certi processi, accoppiando al multithreading e ad una buona pipeline si può migliorare molto la velocità di esecuzione. Questa scelta è stata fatta perché non si riesce a espandere all'infinito la cache o i registri ne si riescono ad aumentare troppo i GigaHertz della CPU, perché si va incontro a consumi elevati e problemi di surriscaldamento (al massimo si arrivare sui 5.5/6 GHz).
Diverso è il discorso del **multicore** in cui anziché avere ad esempio 4 CPU distinte montate sulla scheda madre abbiamo più core (e quindi più ALU) all'interno della stessa CPU, soluzione utilizzata in moltissimi scenari consumer, a differenza invece dei data center che spesso usano anche la tecnica del multiprocessori

Memorie
Memorie che possiamo trovare, dalla più piccola e veloce alla più grande e lenta:

- Registri: memoria velocissima riesce a stare al passo della CPU è molto piccola (nell'ordine di pochi byte) e contiene i dati che servono ai processi per poter essere eseguiti, si trova dentro la CPU
- Cache: memoria anch'essa piccola e veloce posizionata dentro la CPU, a differenza dei registri la cache è più capiente (siamo nell'ordine dei MegaByte, le più grandi arrivano sui 100 mega circa) ma anche più lenta dei registri, restando comunque la

seconda memoria in termini di velocità e dimensione, la cache è divisa in 3 livelli: L1, L2, L3 rispettivamente il più veloce, quello nella media e il più lento, lo scopo principale della cache è evitare in ogni modo che durante la fase di fetch si debba passare per la RAM (cosa che comunque ogni tanto succede) in modo da evitare bottleneck, utilizzando le linee di cache da 64 byte ciascuna (ovvero quando si prendono dei dati dalla RAM si prendono anche i successivi, perché è molto probabile che quei dati saranno utili a breve, quindi il dato prelevato va nei registri mentre quelli successivi aspettano nella cache: cache hit quando quei dati sono serviti veramente e si è risparmiato tempo, cache miss quando quei dati non sono serviti e bisogna andare di nuovo nella RAM)
- RAM, memoria principale dell'architettura di Von Neumann, è veloce anche se molto meno di cache e registri ma in compenso è più capiente (ordine di GigaByte) qui risiedono i dati che servono ai processi per poter essere eseguiti.
- RAM, cache e registri sono memorie volativi ovvero quando il PC viene spento si svuota di tutti i lori dati
- Memoria di massa, questa memoria comprende vari dispositivi, dischi ottici, hard disk, SSD, pen drive ecc..., sono memorie lente rispetto a quelle volatili ma con diverse velocità in base al dispositivo in uso (SSD è più veloce di HDD), sono memorie usate per immagazzinare i dati, quindi sono non volativi ovvero quando il computer si spegne non perdono il loro contenuto
- 

Dispositivi di I/O

Si individuano 2 componenti:

- controller → interfaccia per il SO
- dispositivo in se

Ad esempio la scheda video è un controller per il display. Un controller è quindi un mini calcolatore dotato di una propria unità di elaborazione e una memoria (le scheda video hanno un chip che esegue calcoli e una propria VRAM), usiamo l'esempio di un Hard Disk:
Il controller dell' HDD è fisicamente collegato al disco, il controller si occupa di pilotare il motorino del disco e permette di inviare comandi (motore spento o acceso ad esempio), la CPU da se non riesce a comandare questo controllore, ha quindi bisogno di un driver proprietario, sviluppato dall'azienda dell'HDD che permetta di far dialogare bene SO CPU e controllore con la periferica.
Quando un controller sta lavorando (ad esempio per cercare un file in un HDD), la CPU deve aspettare che arrivi il risultato, ma anziché stare ferma svolge altri compiti, sarà quindi un interrupt mandato dal controller che segnalerà alla CPU che il dispositivo ha fornito la sua risposta